SOLID Principles

A Complete Beginner-Friendly C++ Guide for OOP, LLD and SDE-1 Interviews

What you will learn

- Plain-English explanations of all five principles
- Bad design → improved design examples
- Complete C++ programs from includes to main()
- Expected output and code walkthroughs
- Examples from payments, birds, printers, invoices and service requests
- Interview answers, common mistakes, revision tables and practice questions

Study tip: understand the problem and design decision, not just the code.

How to Use This Guide

Save each complete code example in its own .cpp file. Compile with a C++17 compiler, for example: g++ -std=c++17 main.cpp -o main. Run the executable. Each complete listing is independent; do not paste all listings into one file.

Class vs object: a class defines the shape and behavior; an object is an instance of that class. Inheritance expresses a type relationship. Polymorphism lets a base-class reference call an overridden virtual method on the actual derived object.

Pure virtual function: `virtual void pay() = 0;` makes a class abstract. A concrete derived class must implement it. A virtual destructor is important when deleting derived objects through a base pointer.

Letter	Principle	Question to ask
S	Single Responsibility	Does this class have multiple unrelated reasons to change?
O	Open/Closed	Can I add a new variation without rewriting stable logic?
L	Liskov Substitution	Does a subtype honor the behavior promised by its base?
I	Interface Segregation	Am I forcing classes to implement methods they don't need?
D	Dependency Inversion	Does business logic depend directly on concrete details?

1. S — Single Responsibility Principle (SRP)

Definition: A class should have one cohesive responsibility and one main reason to change. This does not mean one method per class.

Problem: one class does everything

```
#include <iostream>
#include <string>
using namespace std;

class ServiceRequest {
public:
    void create(const string& issue) {
        cout << "Creating request: " << issue << "\n";
    }
    void saveToDatabase() {
        cout << "Saving request to database\n";
    }
    void sendNotification() {
        cout << "Sending notification\n";
    }
};

int main() {
    ServiceRequest request;
    request.create("Flat tyre");
    request.saveToDatabase();
    request.sendNotification();
    return 0;
}
```

The class mixes request behavior, persistence, and messaging. These can change for different reasons.

Improved complete program

```
#include <iostream>
#include <string>
using namespace std;

class ServiceRequest {
    string customer, issue;
public:
    ServiceRequest(string name, string description)
        : customer(name), issue(description) {}

    string summary() const {
        return customer + " needs help: " + issue;
    }
};

class RequestRepository {
public:
    void save(const ServiceRequest& request) const {
        cout << "Saved: " << request.summary() << "\n";
    }
};

class NotificationService {
public:
    void send(const string& message) const {
        cout << "Notification: " << message << "\n";
    }
};

class RequestApplication {
    const RequestRepository& repository;
    const NotificationService& notifications;
public:
    RequestApplication(const RequestRepository& repo,
                       const NotificationService& service)
        : repository(repo), notifications(service) {}

    void submit(const ServiceRequest& request) const {
        repository.save(request);
        notifications.send(request.summary());
    }
};

int main() {
    RequestRepository repo;
    NotificationService notifications;
    RequestApplication app(repo, notifications);
    ServiceRequest request("Rahul", "Flat tyre");
    app.submit(request);
    return 0;
}
```

Output: Saved: Rahul needs help: Flat tyre; Notification: Rahul needs help: Flat tyre.

• `ServiceRequest` represents request data; `RequestRepository` saves it; `NotificationService` sends messages.

• `RequestApplication` coordinates the workflow. Orchestration is also a cohesive responsibility.

Second example: invoice calculation and printing

```
#include <iostream>
#include <iomanip>
using namespace std;

class Invoice {
    double subtotal, taxRate;
public:
    Invoice(double amount, double rate)
        : subtotal(amount), taxRate(rate) {}
    double total() const { return subtotal * (1.0 + taxRate); }
};

class InvoicePrinter {
public:
    void print(const Invoice& invoice) const {
        cout << fixed << setprecision(2)
             << "Invoice total: " << invoice.total() << "\n";
    }
};

int main() {
    Invoice invoice(100.0, 0.18);
    InvoicePrinter printer;
    printer.print(invoice);
    return 0;
}
```

Output: Invoice total: 118.00

Interview answer: SRP says a class should have one main reason to change. Separate responsibilities that evolve independently to improve testing and maintenance.

2. O — Open/Closed Principle (OCP)

Definition: Software should be open for extension but closed for modification. Design stable code so expected new variations can be added with minimal changes to it.

Problem: payment if-else chain

```
#include <iostream>
#include <string>
using namespace std;

class PaymentProcessor {
public:
    void pay(const string& type) const {
        if (type == "UPI") cout << "Paying with UPI\n";
        else if (type == "CARD") cout << "Paying with card\n";
        else if (type == "WALLET") cout << "Paying with wallet\n";
        else cout << "Unsupported payment\n";
    }
};

int main() {
    PaymentProcessor processor;
    processor.pay("UPI");
    processor.pay("CARD");
    return 0;
}
```

Each new payment type requires editing the same conditional. A conditional is not always wrong; a growing list of interchangeable variants is a clue that polymorphism may help.

Improved complete program: payment abstraction

```
#include <iostream>
using namespace std;

class Payment {
public:
    virtual void pay(double amount) const = 0;
    virtual ~Payment() = default;
};

class UpiPayment : public Payment {
public:
    void pay(double amount) const override {
        cout << "UPI: Rs " << amount << "\n";
    }
};

class CardPayment : public Payment {
public:
    void pay(double amount) const override {
        cout << "Card: Rs " << amount << "\n";
    }
};

class WalletPayment : public Payment {
public:
    void pay(double amount) const override {
        cout << "Wallet: Rs " << amount << "\n";
    }
};

void checkout(const Payment& payment, double amount) {
    payment.pay(amount);
}

int main() {
    UpiPayment upi;
    CardPayment card;
    WalletPayment wallet;
    checkout(upi, 500);
    checkout(card, 750);
    checkout(wallet, 200);
    return 0;
}
```

Output: UPI: Rs 500; Card: Rs 750; Wallet: Rs 200.

- `Payment` defines a contract. Derived classes implement their behavior.
- `checkout()` accepts the abstraction and does not change when a new payment implementation is added.

Second example: discounts

```
#include <iostream>
using namespace std;

class Discount {
public:
    virtual double apply(double price) const = 0;
    virtual ~Discount() = default;
};

class NoDiscount : public Discount {
public:
    double apply(double price) const override { return price; }
};

class StudentDiscount : public Discount {
public:
    double apply(double price) const override { return price * 0.90; }
};

class FestivalDiscount : public Discount {
public:
    double apply(double price) const override { return price * 0.80; }
};

int main() {
    StudentDiscount student;
    FestivalDiscount festival;
    cout << student.apply(1000) << "\n";
    cout << festival.apply(1000) << "\n";
    return 0;
}
```

Output: 900 and 800. A new discount rule can be represented by a new implementation.

Interview answer: OCP encourages extension through suitable abstractions and polymorphism rather than repeatedly changing stable conditional logic.

3. L — Liskov Substitution Principle (LSP)

Definition: A subtype should be usable wherever its base type is expected without breaking the behavior promised by that base type. LSP is about behavioral contracts, not merely whether inheritance compiles.

Problem: every bird is assumed to fly

```
#include <iostream>
#include <stdexcept>
using namespace std;

class Bird {
public:
    virtual void fly() const { cout << "Bird flying\n"; }
    virtual ~Bird() = default;
};

class Sparrow : public Bird {
public:
    void fly() const override { cout << "Sparrow flying\n"; }
};

class Penguin : public Bird {
public:
    void fly() const override {
        throw logic_error("Penguins cannot fly");
    }
};

void makeFly(const Bird& bird) { bird.fly(); }

int main() {
    Sparrow sparrow;
    Penguin penguin;
    makeFly(sparrow);
    makeFly(penguin); // Throws: breaks caller's expectation
    return 0;
}
```

The base type promises that all birds can fly, but Penguin cannot fulfill that contract. The inheritance hierarchy models the capability incorrectly.

Improved complete program: separate the capability

```
#include <iostream>
using namespace std;

class Bird {
public:
    virtual void eat() const { cout << "Bird eating\n"; }
    virtual ~Bird() = default;
};

class FlyingBird : public Bird {
public:
    virtual void fly() const = 0;
};

class Sparrow : public FlyingBird {
public:
    void fly() const override { cout << "Sparrow flying\n"; }
};

class Penguin : public Bird {
public:
    void swim() const { cout << "Penguin swimming\n"; }
};

void makeFly(const FlyingBird& bird) { bird.fly(); }

int main() {
    Sparrow sparrow;
    Penguin penguin;
    makeFly(sparrow);
    penguin.eat();
    penguin.swim();
    return 0;
}
```

Output: Sparrow flying; Bird eating; Penguin swimming.

Second example: read-only vs writable objects

```
#include <iostream>
#include <string>
using namespace std;

class Reader {
public:
    virtual string read() const = 0;
    virtual ~Reader() = default;
};

class Writer {
public:
    virtual void write(const string& text) = 0;
    virtual ~Writer() = default;
};

class ReadOnlyFile : public Reader {
public:
    string read() const override { return "existing contents"; }
};

class MemoryFile : public Reader, public Writer {
    string contents;
public:
    string read() const override { return contents; }
    void write(const string& text) override { contents = text; }
};

int main() {
    ReadOnlyFile oldFile;
    MemoryFile file;
    cout << oldFile.read() << "\n";
    file.write("Hello");
    cout << file.read() << "\n";
    return 0;
}
```

Output: existing contents; Hello.

Interview answer: Subclasses must preserve the promises made by their base classes. If a subtype cannot support a promised operation, redesign the contract or model the capability separately.

4. I — Interface Segregation Principle (ISP)

Definition: Clients should not be forced to depend on methods they do not use. Prefer small, cohesive interfaces over one large interface containing unrelated capabilities.

Problem: a basic printer is forced to scan and fax

```
#include <iostream>
using namespace std;

class Machine {
public:
    virtual void print() = 0;
    virtual void scan() = 0;
    virtual void fax() = 0;
    virtual ~Machine() = default;
};

class BasicPrinter : public Machine {
public:
    void print() override { cout << "Printing\n"; }
    void scan() override { cout << "Scan unsupported\n"; }
    void fax() override { cout << "Fax unsupported\n"; }
};

int main() {
    BasicPrinter printer;
    printer.print();
    printer.scan();
    return 0;
}
```

The class must pretend to support operations it cannot perform. Empty methods or exceptions would not fix the interface design.

Improved complete program

```
#include <iostream>
using namespace std;

class Printer {
public:
    virtual void print() = 0;
    virtual ~Printer() = default;
};

class Scanner {
public:
    virtual void scan() = 0;
    virtual ~Scanner() = default;
};

class Fax {
public:
    virtual void fax() = 0;
    virtual ~Fax() = default;
};

class BasicPrinter : public Printer {
public:
    void print() override { cout << "Printing document\n"; }
};

class MultiFunctionPrinter : public Printer, public Scanner, public Fax {
public:
    void print() override { cout << "Printing document\n"; }
    void scan() override { cout << "Scanning document\n"; }
    void fax() override { cout << "Faxing document\n"; }
};

void printDocument(Printer& printer) { printer.print(); }

int main() {
    BasicPrinter basic;
    MultiFunctionPrinter multi;
    printDocument(basic);
    printDocument(multi);
    multi.scan();
    multi.fax();
    return 0;
}
```

Output: Printing document (twice); Scanning document; Faxing document.

- BasicPrinter implements only Printer.
- MultiFunctionPrinter implements all capabilities it supports.

- printDocument() depends only on the Printer interface.

Second example: read and write permissions

A reporting screen may need only read operations, while an admin screen may need read and write operations. Split interfaces where this reflects real client needs; do not force read-only consumers to depend on update/delete methods.

Interview answer: ISP keeps interfaces focused so classes and clients only depend on operations they actually need.

5. D — Dependency Inversion Principle (DIP)

Definition: High-level policy should not depend directly on low-level implementation details. Both should depend on abstractions. Details should implement or plug into those abstractions.

Problem: manager constructs EmailService

```
#include <iostream>
using namespace std;

class EmailService {
public:
    void send() const { cout << "Sending email\n"; }
};

class RequestManager {
    EmailService email; // Hard-coded concrete dependency
public:
    void process() const {
        cout << "Processing request\n";
        email.send();
    }
};

int main() {
    RequestManager manager;
    manager.process();
    return 0;
}
```

The manager is coupled to EmailService. Switching to SMS or using a fake notification during tests requires modifying the manager.

Improved complete program: constructor injection

```
#include <iostream>
#include <string>
using namespace std;

class Notification {
public:
    virtual void send(const string& message) const = 0;
    virtual ~Notification() = default;
};

class EmailService : public Notification {
public:
    void send(const string& message) const override {
        cout << "Email: " << message << "\n";
    }
};

class SmsService : public Notification {
public:
    void send(const string& message) const override {
        cout << "SMS: " << message << "\n";
    }
};

class RequestManager {
    const Notification& notification;
public:
    explicit RequestManager(const Notification& service)
        : notification(service) {}

    void process() const {
        cout << "Processing request\n";
        notification.send("Request received");
    }
};

int main() {
    EmailService email;
    SmsService sms;
    RequestManager emailManager(email);
    RequestManager smsManager(sms);
    emailManager.process();
    smsManager.process();
    return 0;
}
```

Output: Processing request / Email: Request received / Processing request / SMS: Request received.

- Notification is the abstraction; EmailService and SmsService are implementations.
- RequestManager receives the abstraction through its constructor. This is dependency injection.

The injected objects are referenced, not owned. They must outlive the managers that use them.

Second example: customer repository

```
#include <iostream>
#include <string>
using namespace std;

class CustomerRepository {
public:
    virtual bool exists(const string& name) const = 0;
    virtual ~CustomerRepository() = default;
};

class InMemoryRepository : public CustomerRepository {
public:
    bool exists(const string& name) const override {
        return name == "Rahul";
    }
};

class CustomerService {
    const CustomerRepository& repository;
public:
    explicit CustomerService(const CustomerRepository& repo)
        : repository(repo) {}

    void greet(const string& name) const {
        if (repository.exists(name)) cout << "Welcome, " << name << "\n";
        else cout << "Customer not found\n";
    }
};

int main() {
    InMemoryRepository repository;
    CustomerService service(repository);
    service.greet("Rahul");
    service.greet("Aman");
    return 0;
}
```

Output: Welcome, Rahul; Customer not found. The in-memory implementation could be replaced by a database repository without changing the customer-service logic.

Interview answer: DIP reduces coupling by making high-level logic depend on abstractions. Dependency injection is one technique used to implement it.

6. Combined Example: Ride-Resq Request Workflow

This compact example combines SRP, OCP, LSP, ISP and DIP. It is educational code, not a production roadside-assistance system.

```
#include <iostream>
#include <string>
using namespace std;

class ServiceRequest {
    string customer, issue;
public:
    ServiceRequest(string name, string description)
        : customer(name), issue(description) {}
    string summary() const { return customer + " - " + issue; }
};

class RequestRepository {
public:
    void save(const ServiceRequest& request) const {
        cout << "[Repository] Saved " << request.summary() << "\n";
    }
};

class Notification {
public:
    virtual void send(const string& message) const = 0;
    virtual ~Notification() = default;
};

class EmailNotification : public Notification {
public:
    void send(const string& message) const override {
        cout << "[Email] " << message << "\n";
    }
};

class SmsNotification : public Notification {
public:
    void send(const string& message) const override {
        cout << "[SMS] " << message << "\n";
    }
};

class RequestApplication {
    const RequestRepository& repository;
    const Notification& notification;
public:
    RequestApplication(const RequestRepository& repo,
                       const Notification& notifier)
        : repository(repo), notification(notifier) {}

    void submit(const ServiceRequest& request) const {
        repository.save(request);
        notification.send("Request received: " + request.summary());
    }
};

int main() {
    RequestRepository repository;
    EmailNotification email;
    RequestApplication app(repository, email);
    ServiceRequest request("Rahul", "Flat tyre");
    app.submit(request);
    return 0;
}
```

Output: [Repository] Saved Rahul - Flat tyre; [Email] Request received: Rahul - Flat tyre.

Principle	Where it appears
SRP	Request data, repository, notification and application workflow are separated.
OCP	Add a new Notification implementation without changing RequestApplication.
LSP	Each notification implementation must honor the Notification contract.
ISP	Notification exposes one focused capability: send a message.
DIP	RequestApplication depends on Notification, not EmailNotification.

Not every principle is equally visible in every small program. LSP is a behavioral requirement, not something inheritance automatically guarantees.

7. Common Interview Confusions

Confusion	Correct explanation
SRP means one method per class	No. It means one cohesive responsibility and one main reason to change.
OCP bans all edits	No. It aims to reduce repeated edits to stable code when adding expected variations.
Inheritance automatically means LSP	No. A subtype must honor its base class's behavioral contract.
Every interface must have one method	No. Several closely related methods can form a cohesive interface.
DIP and dependency injection are identical	No. DIP is a principle; dependency injection is a technique.
Every class needs an interface	No. Add abstractions where they improve flexibility, boundaries or testing.
More classes always means better design	No. Over-abstraction can make a small system harder to understand.

A method for solving design questions

- 1. Identify the change: What requirement is likely to change?
- 2. Find the coupling: Which class must change when it changes?
- 3. Choose the principle: Responsibility, extension, subtype contract, interface size, or dependency direction?
- 4. Make the smallest useful improvement: Avoid adding abstractions without a clear reason.
- 5. Test the behavior: Ensure the revised design still honors its contract.

8. One-Page Revision Sheet

Principle	Memory phrase	Warning sign	Common improvement
SRP	One main reason to change	Class mixes DB, messaging and business rules	Separate cohesive responsibilities
OCP	Extend stable code safely	Growing if-else for every variant	Abstraction and polymorphism
LSP	Subtypes honor contracts	Subtype breaks expected behavior	Redesign contract or capability
ISP	Focused interfaces	Irrelevant required methods	Split capabilities
DIP	Depend on abstractions	Business logic creates concrete infrastructure	Inject an abstraction

Five short interview answers

SRP: A class should have one main responsibility and one main reason to change.

OCP: New behavior should be added through extensions with minimal changes to stable logic.

LSP: Subtypes must preserve the behavior promised by their base types.

ISP: Clients should not depend on methods they do not use.

DIP: High-level logic should depend on abstractions, not concrete implementation details.

Keywords: cohesion • coupling • abstraction • polymorphism • behavioral contract • capability • constructor injection • testability

9. Practice Questions with Suggested Answers

1. A Report class reads SQL data, calculates metrics, formats a PDF and emails it. Which principle is at risk?

2. Your checkout function has a branch for every payment provider. How can OCP help?

3. A read-only file inherits from a writable-file class and throws from write(). Why is that an LSP concern?

4. BasicPrinter is forced to implement scan() and fax(). Which principle applies?

5. OrderService constructs EmailService internally. How could DIP reduce coupling?

6. Does creating five classes automatically mean a design follows SOLID?

7. Can an interface have several methods and still follow ISP?

8. Which principle is concerned with safely substituting a subtype for its base type?

Answers

1. SRP. Separate data access, calculation, PDF presentation and email delivery into cohesive components.

2. Define a payment abstraction and implement each provider behind it; checkout uses the abstraction.

3. The base promises writing, but the subtype cannot honor it. Split read/write capabilities or redesign the contract.

4. ISP. Split printing, scanning and faxing into focused interfaces.

5. Define a notification abstraction and inject an implementation through the constructor.

6. No. SOLID concerns responsibility, extension, contracts, interfaces and dependency direction—not class count.

7. Yes. Several closely related methods can make one cohesive interface.

8. LSP.

Checklist before moving to design patterns

- ☐ I can explain all five principles in my own words.
- ☐ I can identify a violation in a short code example.
- ☐ I can rewrite the example into a better design.
- ☐ I understand pure virtual functions, overriding and references.
- ☐ I can explain the trade-off and avoid unnecessary abstractions.

End of guide • Revise the examples, then implement the improved versions without looking at the code.